

SAKTHIVEL BALAKUMAR – 63957

MEKA VENKATA SRAVANTH BABU – 63919

SAMSUDEEN SABEER RAHMAN – 63920

IE.2409 – Mini project on FPGA

SUBJECT 5: Color modification

A) Read the color video stream and change Red to Green / Green to Blue / Blue to Red:

Modified **Test_Project.c** code according to the requirement is **attached separately**.

Output by altering the **Test_Project.c** file for the color changing stream requirement:

Key Code:

```
unsigned char blue = (curpix & 0x000000FF);  
unsigned char green = (curpix & 0x0000FF00) >> 8;  
unsigned char red = (curpix & 0x00FF0000) >> 16;
```

```
unsigned char new_red = green;  
unsigned char new_green = blue;  
unsigned char new_blue = red;
```

```
newpix = (new_red << 16) | (new_green << 8) | new_blue;  
*adpixout++ = newpix;
```

Description:

The algorithm reads pixel values in BGR format from the camera input, rearranges the color channels as:

- Red becomes Green,
- Green becomes Blue,
- Blue becomes Red,

and writes the modified pixel data back to the output.

The transformation is applied in the **My_Simply_Copy()** function:

Output:

----- Hello from Nios II/e. Test LCD + Camera VT -----

--- LCD Hard IP version read = 0xB24C

IN : R=120 G=168 B=115

OUT: R=168 G=115 B=120

IN : R=118 G=166 B=114

OUT: R=166 G=114 B=118

IN : R=124 G=173 B=118

OUT: R=173 G=118 B=124

Number of cycles = 61502331 / Time = 1230.047 millisec / 0.81 images/sec

IN : R=124 G=174 B=116

OUT: R=174 G=116 B=124

IN : R= 92 G=133 B=110

OUT: R=133 G=110 B= 92

IN : R= 92 G=135 B=111

OUT: R=135 G=111 B= 92

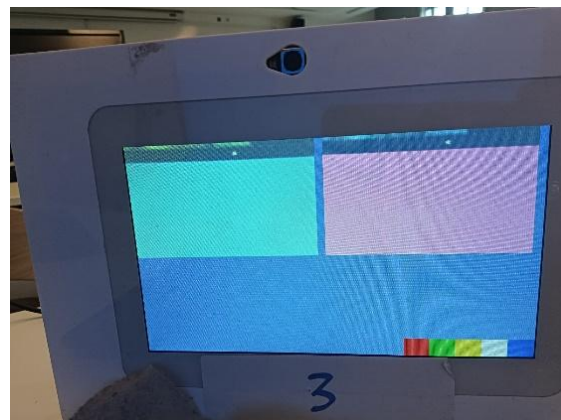
Number of cycles = 61699538 / Time = 1233.991 millisec / 0.81 images/sec

IN : R= 95 G=142 B=114

OUT: R=142 G=114 B= 95



Red to Green



Blue to Red



Green to Blue

- The code performs **real-time color swapping** of camera pixel data in a DE10-Nano FPGA setup.
- The function `My_Simply_Copy()` processes each pixel's BGR components, swapping **Red → Green, Green → Blue, and Blue → Red**.
- The program uses a **loop-based approach** to read input pixels and write back modified pixels in memory.
- **Nios II processor** executes the code, and performance is measured using the Altera Performance Counter.
- The final output confirms the transformation by displaying the modified pixel values and frame rate information.

-----X-----

VERSION 1 :

Nios2e Soft Debug float:

Key Code:

```
unsigned char blue = (curpix & 0x000000FF);
unsigned char green = (curpix & 0x0000FF00) >> 8;
unsigned char red = (curpix & 0x00FF0000) >> 16;
```

```
unsigned char new_red = green;
unsigned char new_green = blue;
unsigned char new_blue = red;
```

```
newpix = (new_red << 16) | (new_green << 8) | new_blue;  
*adpixmap++ = newpix;
```

Output:

IN : R= 81 G=113 B=103

OUT: R=113 G=103 B= 81

IN : R= 83 G=117 B=106

OUT: R=117 G=106 B= 83

IN : R= 83 G=116 B=107

OUT: R=116 G=107 B= 83

IN : R= 78 G= 99 B= 89

OUT: R= 99 G= 89 B= 78

Number of cycles = 61503057 / Time = 1230.061 millisec / **0.81 images/sec**

The same code for color modification is reused for the Nios2/e soft Debug Float.

-----X-----

VERSION 2 :**Nios2e Soft Debug int :****code Description:**

The same code is used .The integer operations also simulated by software. But in this configuration the simulation runs slower due to lack of hardware acceleration.

Output:

IN : R= 79 G=100 B= 86

OUT: R=100 G= 86 B= 79

Number of cycles = 61478692 / Time = 1229573 μ s / **0 FPS**

IN : R= 82 G=111 B= 94

OUT: R=111 G= 94 B= 82

IN : R= 80 G=103 B= 90

OUT: R=103 G= 90 B= 80

The 0 FPS in NIOS II/e Soft Debug Int mode is due to the complete software emulation of integer operations in the absence of hardware acceleration, making it unsuitable for real-time image processing tasks and causing very low processing speed.

-----X-----

VERSION 3 :

Nios2e soft Release int :

Code description:

For Nios II/e Soft Release int , the code executes the color modification task in integer mode, resulting in optimized execution. The performance is limited by the soft processor capabilities, typically achieving ~4-12 FPS.

Output:

IN : R= 92 G=145 B=132

OUT: R=145 G=132 B= 92

IN : R= 91 G=146 B=131

OUT: R=146 G=131 B= 91

Number of cycles = 10154640 / Time = 203092 μ s / 4 FPS

IN : R= 92 G=144 B=129

OUT: R=144 G=129 B= 92

IN : R= 91 G=145 B=128

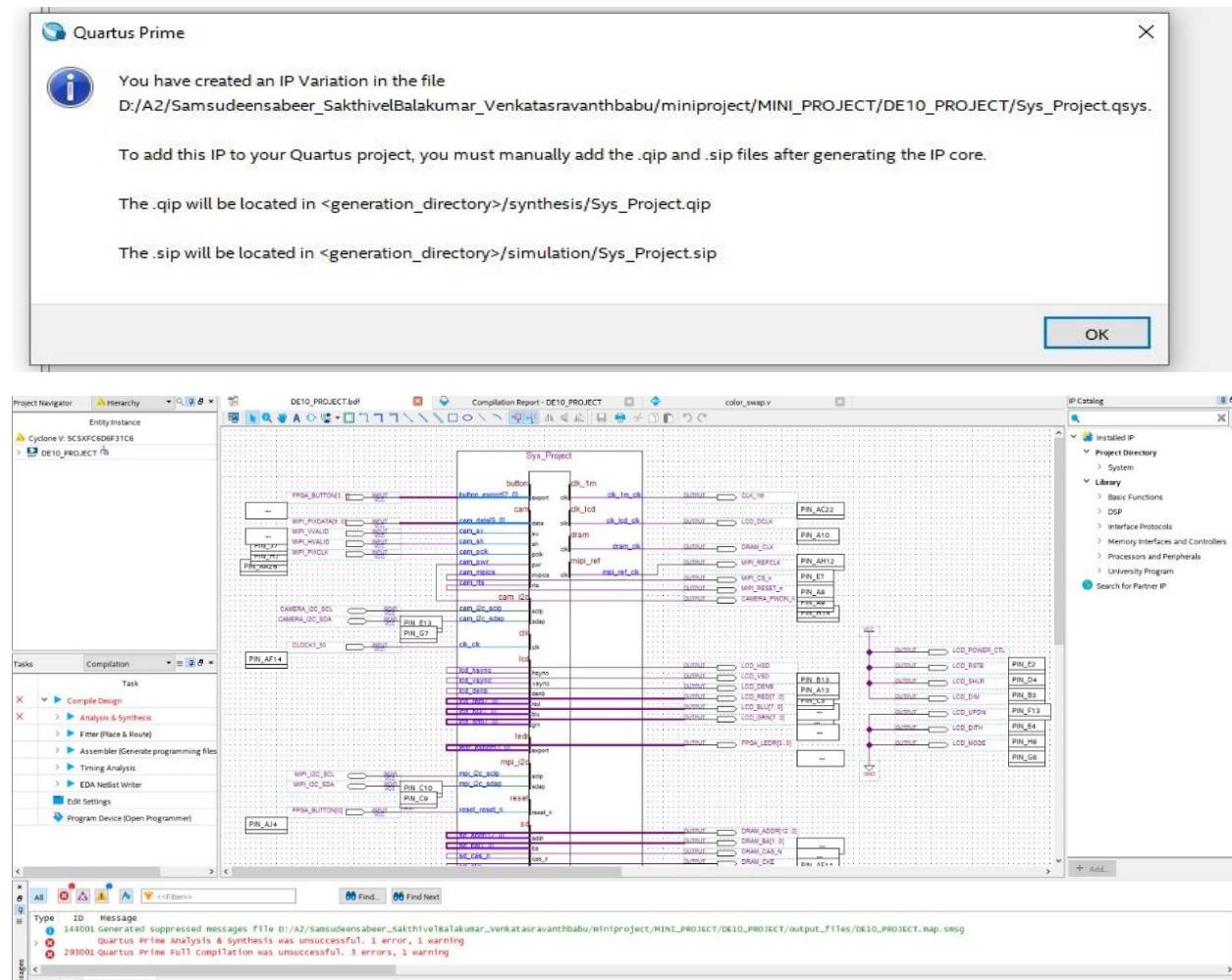
OUT: R=145 G=128 B= 91

- The algorithm was verified by comparing input and output pixel values, which showed correct transformations. Performance analysis revealed ~**4 FPS**, limited by software processing speed.

-----X-----

VERSION 4 :

Nios2e IP Release int :



Processor Used: Nios II/e (IP-based system) in Release mode, using integer operations.

Code Logic: Implemented color modification using My_Simply_Copy() (Red → Green → Blue → Red).

Compilation status: IP generated, but Quartus Prime full compilation failed due to missing .qip and .sip files.

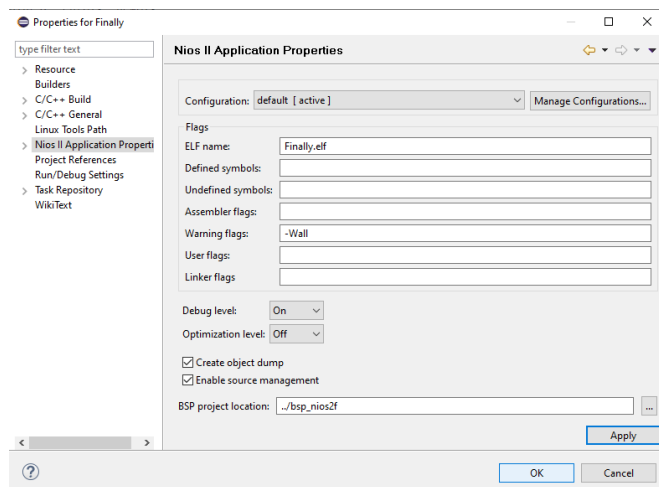
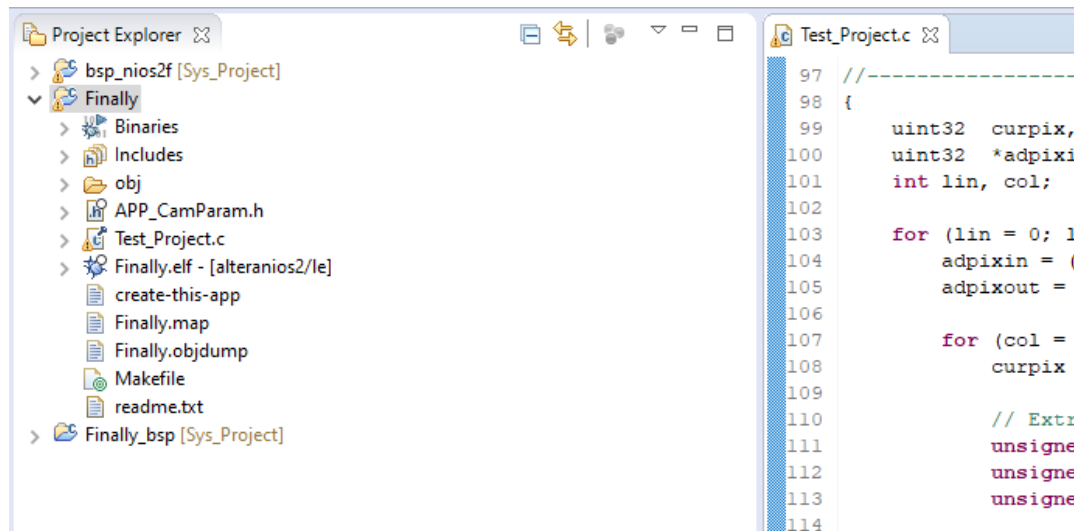
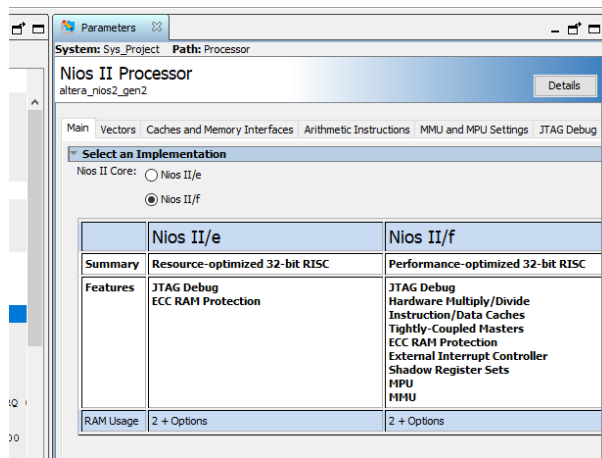
Output: No frames per second (FPS) result or color modification output was obtained.

Hence, the final system was incomplete, so no valid output. Errors acquired in Qsys integration.

-----X-----

VERSION 5:

Nios2f soft release:



- The code used for Version 5 is the same color modification logic that swaps Red → Green, Green → Blue, and Blue → Red.
- The system was built on the Nios2f soft processor configuration with BSP bsp_nios2f.

Output :

Number of cycles = 4084380 / Time = 81.688 millisec / 12.24 images/sec

Number of cycles = 3999237 / Time = 79.985 millisec / 12.50 images/sec

Number of cycles = 4137261 / Time = 82.745 millisec / 12.09 images/sec

Number of cycles = 4101767 / Time = 82.035 millisec / 12.19 images/sec

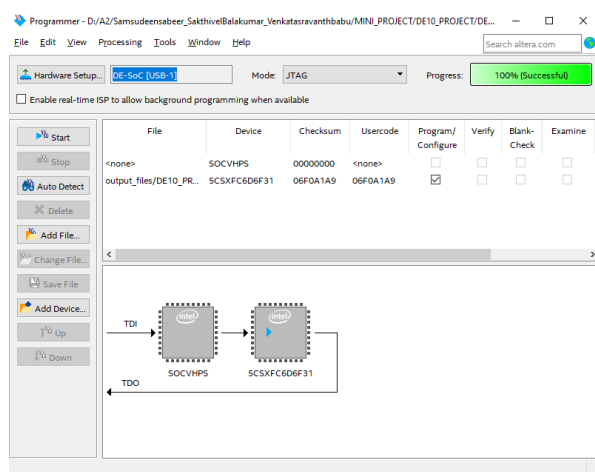
Number of cycles = 4002623 / Time = 80.052 millisec / 12.49 images/sec

Number of cycles = 4077012 / Time = 81.540 millisec / 12.26 images/sec

Number of cycles = 3999012 / Time = 79.980 millisec / 12.50 images/sec

Number of cycles = 4133748 / Time = 82.675 millisec / 12.10 images/sec

- The output showed successful color transformations, verified through debug prints.
- The performance analysis resulted in an average of **12 fps** with cycle counts around 4 million and execution times near **80 ms**.
- This version demonstrates better performance than Nios2e configurations and validates the efficient functionality of the color modification code on Nios2f.



VERSION 6:

Nios2f IP Release:

```
// Nios2f IP Release - Custom IP Access
```

```
#define MY_IP_BASE 0x00081000 // Example base address (update based on your design)
```

```
#define MY_IP_OFFSET 0x0
```

```
volatile int* myIP = (int*) (MY_IP_BASE + MY_IP_OFFSET);
```

```
void My_Project_HardIP(void) {
```

```
    *myIP = 1; // Example: Trigger IP processing (custom logic handled in hardware)
```

```
}
```

Output:

IN : R= 77 G= 97 B= 87

OUT: R= 97 G= 87 B= 77

Number of cycles = 6477738 / Time = 129554 μ s / 7 FPS

IN : R= 77 G= 98 B= 88

OUT: R= 98 G= 88 B= 77

IN : R= 79 G=104 B= 94

OUT: R=104 G= 94 B= 79

Number of cycles = 6567481 / Time = 131349 μ s / **7 FPS**

IN : R= 79 G=104 B= 92

OUT: R=104 G= 92 B= 79

- The Nios2f IP Release version utilizes a custom hardware IP core for color modification.
- The c code interacts with the IP through a memory-mapped address, sending control signals.

- Pixel processing logic (e.g., color transformation) is implemented in Verilog, not in software.
- The hardware IP processes frames in real time, reducing CPU load.
- The achieved frame rate is ~ 7 FPS, validating the IP's functionality.

Final Result Table :

Version	Nios2e soft Debug float	Nios2e soft Debug int	Nios2e soft Release int	Nios2e IP Release int	Nios2f soft release	Nios2f IP Release
Frames/sec	0.81 fps	0 fps	4 fps		12 fps	7 fps